

Masaüstü Uygulama Geliştirme

Hafta-13

Öğr. Gör. Zafer SERİN

Sanal Yapılar Virtual & Override

- Bir nesne üzerinde var olan tüm memberlar(üyeler) derleme zamanında belirgindir.
- Yani, derleme aşamasında hangi nesne üzerinden hangi metotların çağrılacağı bilinmektedir.

```
class MyClass
{
    0 references
    public int MyProperty { get; set; }
    0 references
    public void MyMethod() { }
}
```

- Derleme zamanından olaya bakıldığında MyClass sınıfından üretilen tüm nesnelerde MyPproperty ve MyMethod çağrılacaktır.

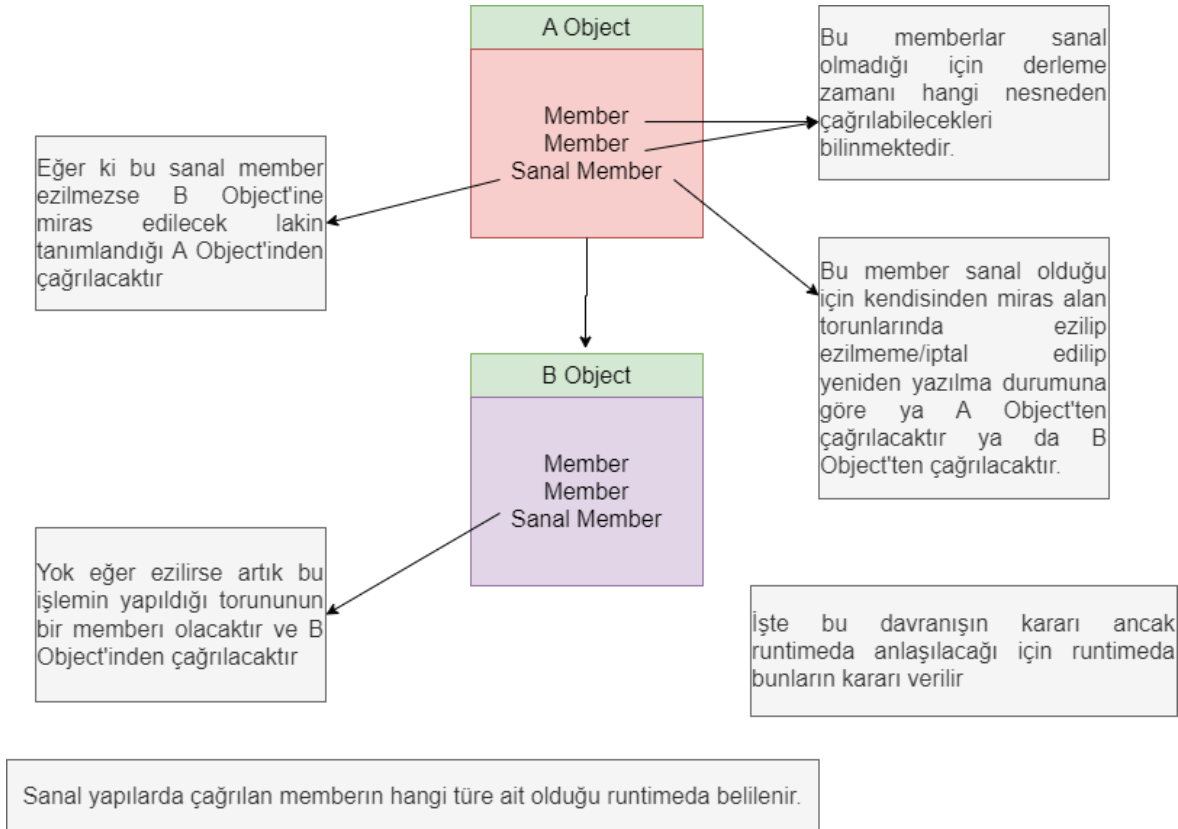
Sanal Yapılar Virtual & Override

- Sanal yapılar ile derleme zamanında kesin olarak bilinen bu bilgi runtime(çalışma zamanı) için de belirlenebilmektedir. Yani ilgili nesnenin hangi metodu kullanacağı runtime'da kararlaştırılır.
- Sanal yapılar, bir sınıf içerisinde bildirilmiş olan ve o sınıftan türeyen alt sınıflarda da tekrar bildirilebilen yapılardır.
- Bu yapılar metod, property veya indexer olabilir.
- Sanal yapılanmalarda üstten gelen bir yapının işlevi iptal edilip yeniden yapılandırılır.

Sanal Yapılar Virtual & Override

- Bir sınıfta tasarlanmış sanal yapının işlevinin iptal edilip edilmeme durumuna göre tanımlandığı sınıftan mı yoksa bu sınıfın torunlarından mı çağrılacağına belirlenmesi runtime da gerçekleşecektir.

Sanal Yapılar Virtual & Override



Sanal Yapılar Ne için Kullanılır?

- Bir sınıfta tanımlanmış olan herhangi bir memberın kendisinden türeyen alt sınıflarda ezilmesi için kullanılır.
- Peki bu zorunlu mudur? Yani bir sanal yapı illaki kendisinden türeyen torunlarda ezilmek zorunda mıdır?
- Tabiki değildir. Yani bir member sanal yapıldı diye illa ki kendisinden türeyen alt sınıflarda ezilmek zorunda değildir! Ama ezilmek istenirse direkt ilgili sınıfın bir memberı olacak şekilde çalışılmasını sağlamış olur.

Bir Sınıfta Sanal Yapı Nasıl Oluşturulur?

- Bir sınıfta sanal yapılar oluşturabilmek için ilgili memberın imzasını **virtual** keywordü ile işaretlemek yeterlidir.
- public virtual ya da virtual public şeklinde kullanılabilir.

Bir Sınıfta Sanal Yapı Nasıl Oluşturulur?

- Bir sınıfta sanal yapılar oluşturabilmek için ilgili memberın imzasını **virtual** keywordü ile işaretlemek yeterlidir.
- public virtual ya da virtual public şeklinde kullanılabilir.

```
class MyClass
{
    // Normal metot ve property tanımlama
    0 references
    public int MyProperty { get; set; }
    0 references
    public void MyMethod() { }
}
```

```
class MyClass
{
    // Sanal metot ve sanal property tanımlama
    0 references
    public virtual int MyProperty { get; set; }
    0 references
    virtual public void MyMethod() { }
}
```

Bir Sınıfta Sanal Yapı Nasıl Oluşturulur?

- Bir class'ta virtual ile işaretlenerek sanal hale getirilmiş bir member, bu classtan miras alan torunlarında ezilmek isteniyorsa eğer ilgili classta imzası **override** keywordü ile işaretlenmiş bir vaziyette tekrardan aynı member oluşturulur.

Bir Sınıfta Sanal Yapı Nasıl Oluşturulur?

```
class MyClass
{
    1 reference
    virtual public void MyMethod() { }
}
```

0 references

```
class MyClass2 : MyClass
{
    1 reference
    public override void MyMethod()
    {
    }
}
```

- Base classta virtual ile işaretlenen member ezilmek zorunda değil; ancak bir torun class base classtaki herhangi bir memberı ezecekse o member mutlaka virtual ile işaretlenmiş olmalı!

Bir Sınıfta Sanal Yapı Nasıl Oluşturulur?

- Bir classta virtual ile işaretlenmiş herhangi bir member illa ki direkt o classtan türeyen 1. dereceden classlarda override edilmek zorunda değil! İhtiyaca göre alt seviyedeki torunlardan herhangi birinde override edilebilir.
- Ancak böyle bir durumda virtual member en son override edildiği Object'ten sonra geçerli olur.

Çok Biçimlilik(Polimorfizm) Nedir?

- Polimorfizm esasında kalıtım gibi bir biyolojik terimdir.
- Polimorfizm biyolojik açıdan iki veya daha fazla farklı fenotipin aynı tür popülasyonunda bulunmasıdır.
- Polimorfizm yazılımda iki veya farklı nesnenin aynı tür classlardan(referanslardan) referans almasıdır.(işaretlenebilmesidir).
- Bir diğer tanımda Polimorfizm bir nesneyi birden fazla form ile tarif edebilmektir.

Çok Biçimlilik(Polimorfizm) Nedir?

- OOP'de polimorfizm, iki ya da daha fazla nesnenin aynı tür sınıf tarafından karşılanabilmesi/referans edilebilmesidir.
- Bir başka deyişle, bir nesnenin birden fazla farklı türdeki referans tarafından işaretlenebilmesi polimorfizmdir.
- Bir başka tanım Polimorfizm, bir nesnenin kalıtımsal olarak ilişkisi olan diğer nesnelerin referanslarıyla işaretlenebilmesini sağlar.

Çok Biçimlilik(Polimorfizm) Nedir?

- Polimorfizm, OOP tasarımlarında geliştirilen koda daha manevrasal bir şekilde nitelik kazandıran ve yaklaşım sergilememizi sağlayan bir özelliktir.
- Polimorfizm, programlamada ki temel prensiplerden olan 'Sol/Sağ' prensibini aşır, eldeki nesnenin birden fazla referans ile işaretlenebilmesini sağlar.

Çok Biçimlilik(Polimorfizm) Nedir?

- Bir nesnenin birden fazla referansla işaretlenmesi; o nesnenin, birden fazla türün davranışını gösterebilmesini sağlar.

```
class Test{}  
0 references  
internal class Program  
{  
    0 references  
    static void Main(string[] args)  
    {  
        Test t1 = new Test();  
    }  
}
```

Çok Biçimlilik(Polimorfizm) Nedir?

- Kuş deyince aklımıza kartal, papağan, deve kuşu, tavus kuşu vb. gelebilmektedir. Bunların hepsi temelde bir kuştur. Bununla birlikte kartal hem kuş özelliklerine hem de kartal özelliklerine sahip yani çok biçimlidir.
- Üstteki maddeye göre ortak atadan gelen, kalıtımsal olarak 'Kuş' tan türeyen tüm hayvanlar kendi türleri yahut 'Kuş' türü ile referans edilebilirler.
- Buradan da şunu anlıyoruz ki, yazılımsal açıdan çok biçimliliğin söz konusu olabilmesi için teknik olarak Kalıtım olması gerekmektedir!

Programlama da Polimorfizm Nerede Kullanılır?

- Bilinçsiz tür dönüşümü,
- Object türünün herhangi türdeki bir değeri alabilmesi,

Soyutlama(Abstraction)

- Soyutlama bir mantıktır, bir davranıştır.
- Tanım olarak soyutlama bir iş yaparken sadece ihtiyaç duyulan yapıların bulunması durumudur.
- Programlama açısından kullanıcı işlemleri için kullanılacak bir classta yüzlerce member olabilir, buna karşın sadece kullanıcı kayıt etmek isteyen bir kişiye bu metodun gösterilmesi yeterli olacaktır. İşte bu soyutlamadır.
- Bir başka deyişle soyutlama gerekli olanları göster, gereksiz olanları gösterme demektir.

Soyutlama(Abstraction)

- Soyutlama ile bir sınıfın memberlarından ihtiyaç noktasında alakalı olanları gösterip, alakalı olmayanları göstermemektir.

Abstract Class

- Abstract Class kullanılarak abstraction uygulanabilir.
- Abstraction = abstract class şeklinde bir düşünce yanlıştır. Bunlar aynı kelime kökenine sahip olmasına karşın abstraction interface ile de uygulanabilir. Burada dikkat edilmesi gereken abstraction davranışının abstract class ile uygulanabilecek olmasıdır.
- Bir sınıfın uyması gereken temel yapıyı tanımlamak için abstract class yapısı kullanılabilir ve gerekli modellemeyi gerçekleştirebilirsiniz.

Abstract Class

- Abstract classlardan normal şartlarda nesne üretilemez!

Interface(Arayüz) Nedir?

- Programlama süreçlerinde Interface yapılanması, nesnelere direkt olarak bir arayüz/şablon oluşturulmasını ve bu arayüz üzerinden geliştirici ile nesne arasındaki etkileşimin daha da kolaylaştırılmasını sağlayan bir araçtır.
- Hatta sadece geliştirici ile nesne arasındaki süreci kolaylaştırmamakta, ayrıca bir programın farklı bir programla veya bileşenle etkileşimini de kolaylaştırmaktadır.

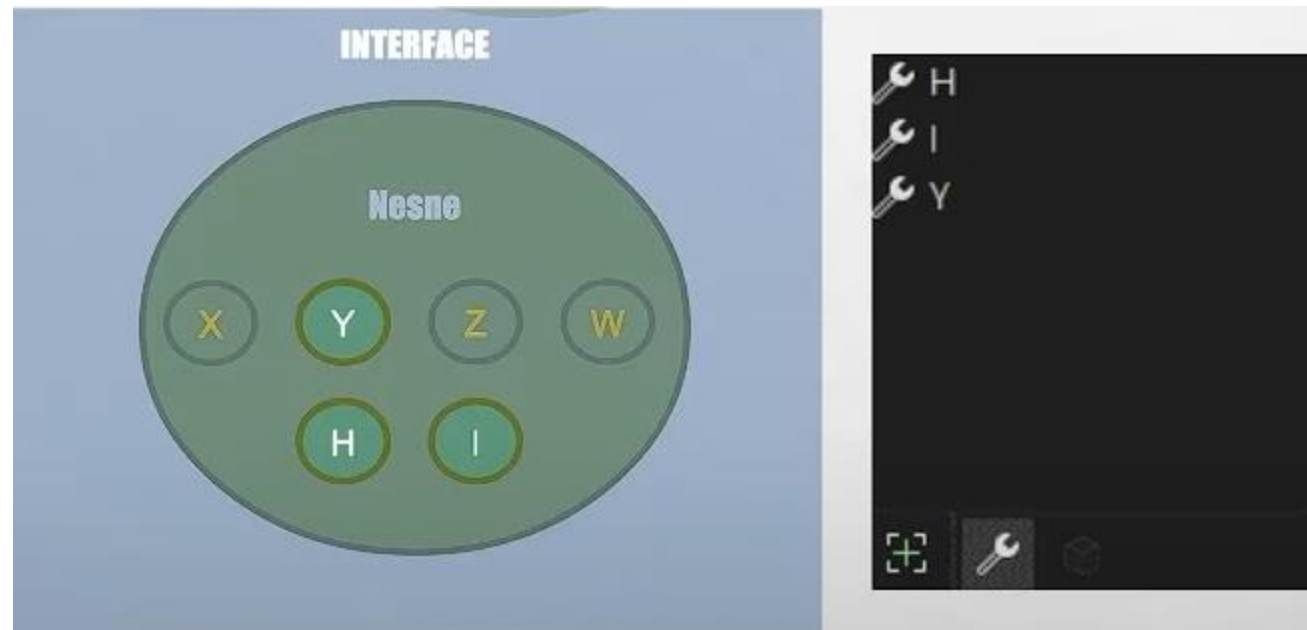
Interface(Arayüz) Nedir?

- Interface'in nesneye bir arayüz sağlaması, kullanıcı açısından ilgili nesnenin nasıl çalıştığına dair ayrıntılı bilgiye ihtiyaç duyulmaksızın, sadece arayüzün sunduğu fonksiyonların veya propertyleri kullanarak etkileşime girilmesini sağlar.
- Interface ilgili nesneye abstraction(soyutlama) uygulayarak belirlenmiş interface üzerinden çalışılmasını ve böylece ilgili nesne ile geliştirme sürecinin kolaylaştırılmasını sağlamaktadır.

Interface(Arayüz) Nedir?

- Abstract classlarda bir nesnenin içerisindeki imzaları modellemenin dışında farklı işlemlerde gerçekleştirilebilirken interfacerler sadece imzalara odaklanan bir yapıdır denilebilir.
- **Interface, abstract classın sadece imzalara konsantre olmuş halidir! O yüzden abstraction(soyutlama) davranışı açısından abstract classa nazaran daha elverişlidir!**

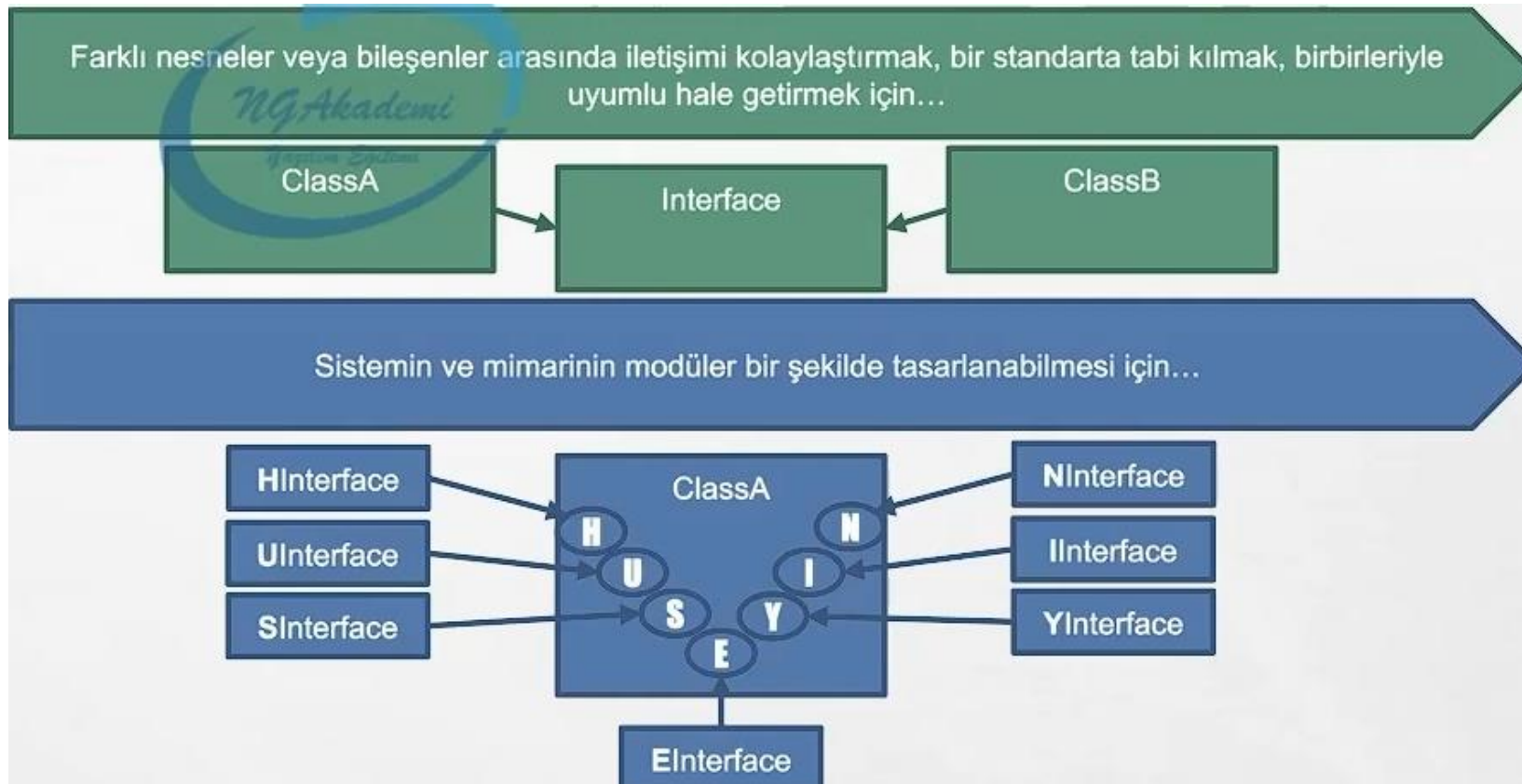
Interface(Arayüz) Nedir?



Interface(Arayüz) Nedir?

- Bizler bir sınıfta(classta) olmasını istediğimiz memberları oluşturmada önce bir interfacede bu memberların imzalarını tanımlayarak bir şablon oluşturuyoruz.
- Ardından bu interface i kullanarak sınıfta memberları tanımlıyoruz.
- Bir sınıf, bir interface kullanıyorsa eğer aynen abstract classlardaki abstract elemanlarda olduğu gibi içerisindeki memberların tanımlanmasını zorunlu kılar!

Neden Interface Kullanırız?



Interface Nasıl Tanımlanır?

- Interface tanımlayabilmek için 'interface' anahtar sözcüğünden faydalanırız.
- Interface'ler referans türlü değişkenlerdir.
- C# programlama dilinde bir Interface'e isim verirken 'I' ile başlaması bir gelenektir.
- Bir class nerede tanımlanabiliyorsa interface te orada tanımlanabilir.

Interface Nasıl Tanımlanır?

```
interface IKullanici  
{  
  
}
```

Interface içerisinde İmzaların Oluşturulması

- Interface içerisinde metot ve property imzaları tanımlanabilmektedir.
- İmzalar tanımlanırken erişim belirteci kullanılmaz, gövdeler tanımlanmaz sadece imzalar tanımlanacaktır.
- Abstract classlarda olduğu gibi imzalarda 'abstract' keywordü kullanılmaz! Çünkü interfaceler sadece sınıflara uygulanılacak imzaları barındırır abstract classlarda olduğu gibi normal memberlara sahip değildir!
- Interface içerisinde field tanımı mümkün değildir!

Interface içerisinde İmzaların Oluşturulması

```
interface IKullanici
{
    void Dogrula();
    int Hesapla();
    int Yas { get; set; }
}
```

Interface Bir Class'a Nasıl Uyguladır

- Interfacei kullanabilmek için kalıtım sürecinde kullanılan ':' operatörü kullanılır, ancak bu işlemin adı artık kalıtım değil implementasyon olur! Aynı durum abstract class içinde geçerlidir.
- Interfacelerden yapılan implementasyon sonucu ilgili class interfaceteki imzaları uygulamak zorundadır ancak abstract classlarda olduğu gibi bu işlem için 'override' anahtar sözcüğü kullanılmaz!
- Interfaceler sınıfların imzasıdır!

Interface Bir Class'a Nasıl Uyguladır

```
interface IPersonel  
{  
    void Dogrula();  
}
```

```
class Personel : IPersonel  
{  
    public void Dogrula()  
    {  
    }  
}
```

Interfacelerde Çoklu Kalıtım Durumu

- C#'ta bir class sadece tek bir classtan kalıtım alabilmektedir!
- Buna karşın bir class birden fazla Interfacei aynı anda uygulayabilmektedir.
- Ayrıca ilgili classın nesnesi tüm bu interfaceler ile refere edilebilmekte ve soyutlama davranışı gerçekleştirilebilmektedir.

Interfacelerde Çoklu Kalıtım Durumu

```
interface ITopla
{
    int Topla();
}

interface ICarp
{
    int Carp();
}

class Islem : ITopla, ICarp
{
    public int Topla()
    {
        return 5;
    }
    public int Carp()
    {
        return 5;
    }
}
```

Interfaceler Birbirinden Kalıtım Alabilir mi?

- Interfaceler kendi aralarında kalıtım alabilir. Bu işlem implementasyon olarak değil normal kalıtım olarak isimlendirilir. Burada standart kalıtım kuralları geçerli olacaktır. Buradaki en önemli fark Interfaceler kendi aralarında kalıtım uygulasa bile yatayda yine birden çok Interface uygulanabilir!

Interfaceler Birbirinden Kalıtım Alabilir mi?

```
interface ITopla  
{  
    int Topla();  
}
```

```
interface ICikar  
{  
    int Cikar();  
}
```

```
interface ICarp : ITopla, ICikar  
{  
    int Carp();  
}
```

Hem Kalıtım Hem Implementasyon Uygulanabilir mi?

- C#'ta bir class, farklı bir classtan kalıtım alırken hem de bir yandan interfacei implemente edecekse eğer önce classtan kalıtım almalı, sonrasında da interfaceler tanımlanmalıdır. Buradaki öncelik önemlidir aksi taktirde derleyici hata verecektir.

Hem Kalıtım Hem Implementasyon Uygulanabilir mi?

```
class Canli
{

}

interface ISolunum
{
    void SolunumYap();
}

interface IDolasim
{
    void DolasimYap();
}

class Insan : Canli, ISolunum, IDolasim
{
    public void DolasimYap()
    {
    }
    public void SolunumYap()
    {
    }
}
```